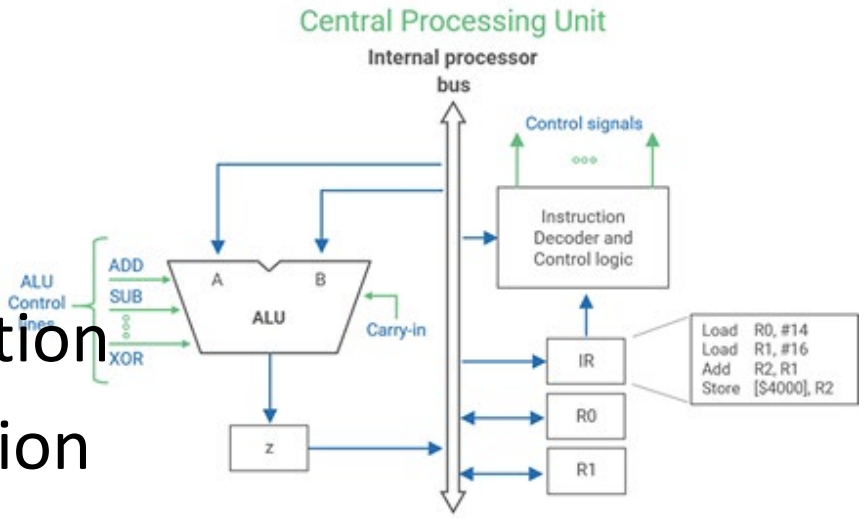
Lecture 8

Pipelining

Previously..

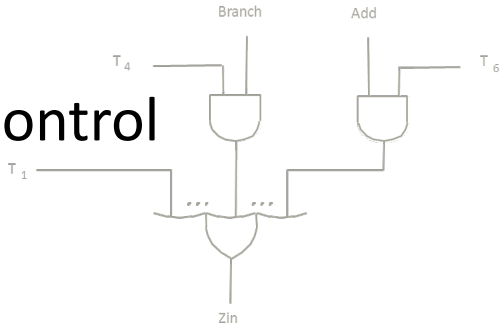
- Basic of Executing an Instruction
- Step of Executing an Instruction
  - Register transfers data
  - Fetches the data
  - Performs an arithmetic or a logic
  - Stores a word
- Stage and Datapath
- Decode/Encode instruction and Control logic
  - Hardwired control
  - Microprogrammed control



Ex. Add (R3), R1

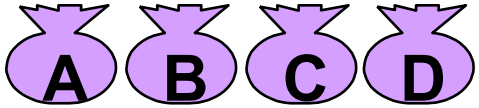
Step	Action
1	PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2	Z_{out}, PC_{in}, WMFC
3	MDR_{out}, IR_{in}
4	$R3_{out}$, MAR_{in}, Read
5	$R1_{out}$, Y_{in}, WMFC
6	MDR_{out}, SelectY, Add, Z_{in}
7	Z_{out}, $R1_{in}$, End

Understand Stages and Data Paths

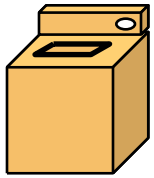


Traditional Pipeline Concept (1)

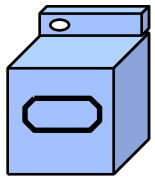
- Laundry Example
- Alice, Bob, Cathy, Dave each have one load of clothes to wash, dry, and fold



- Washer takes 30 minutes



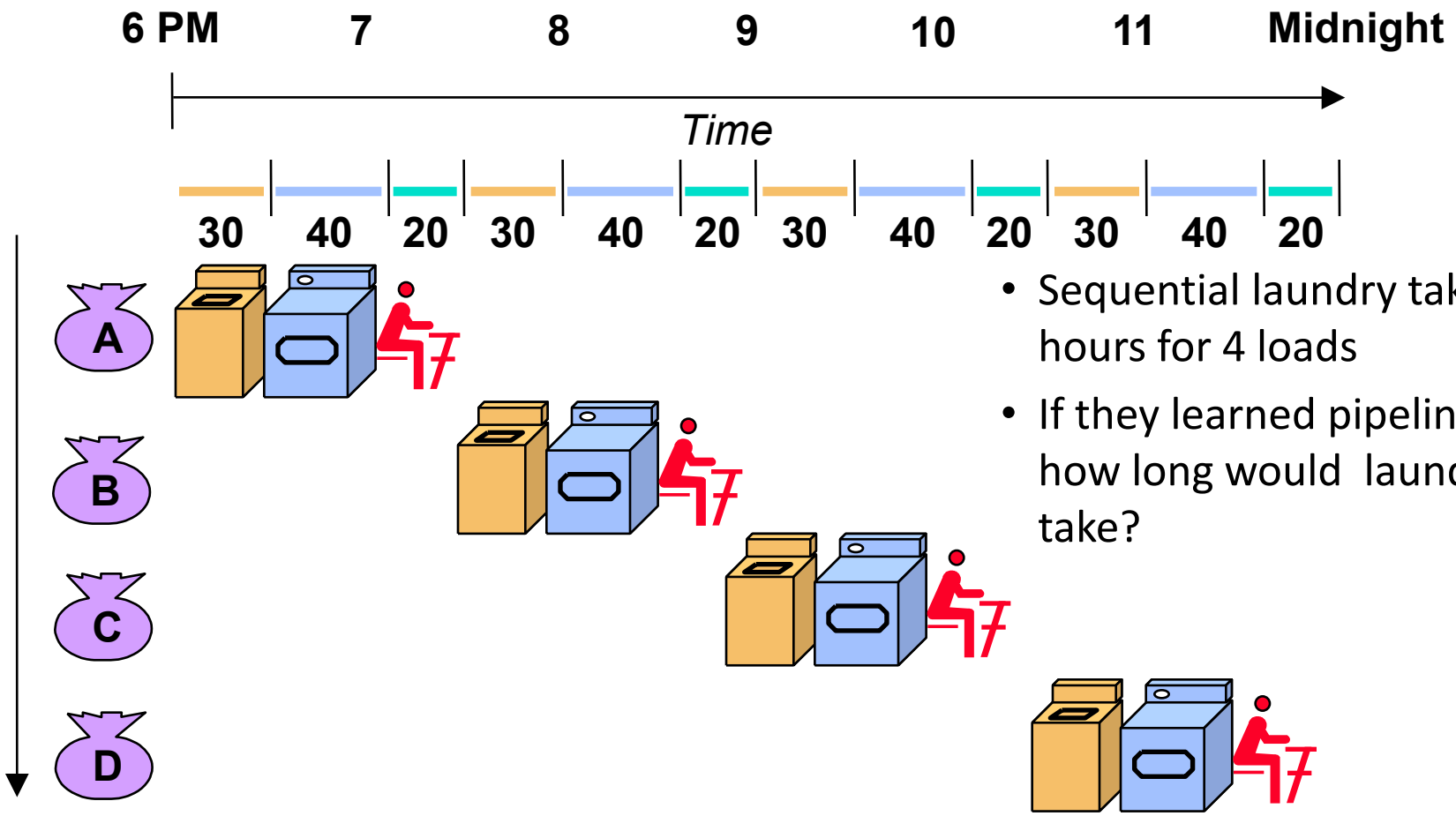
- Dryer takes 40 minutes



- Folder takes 20 minutes

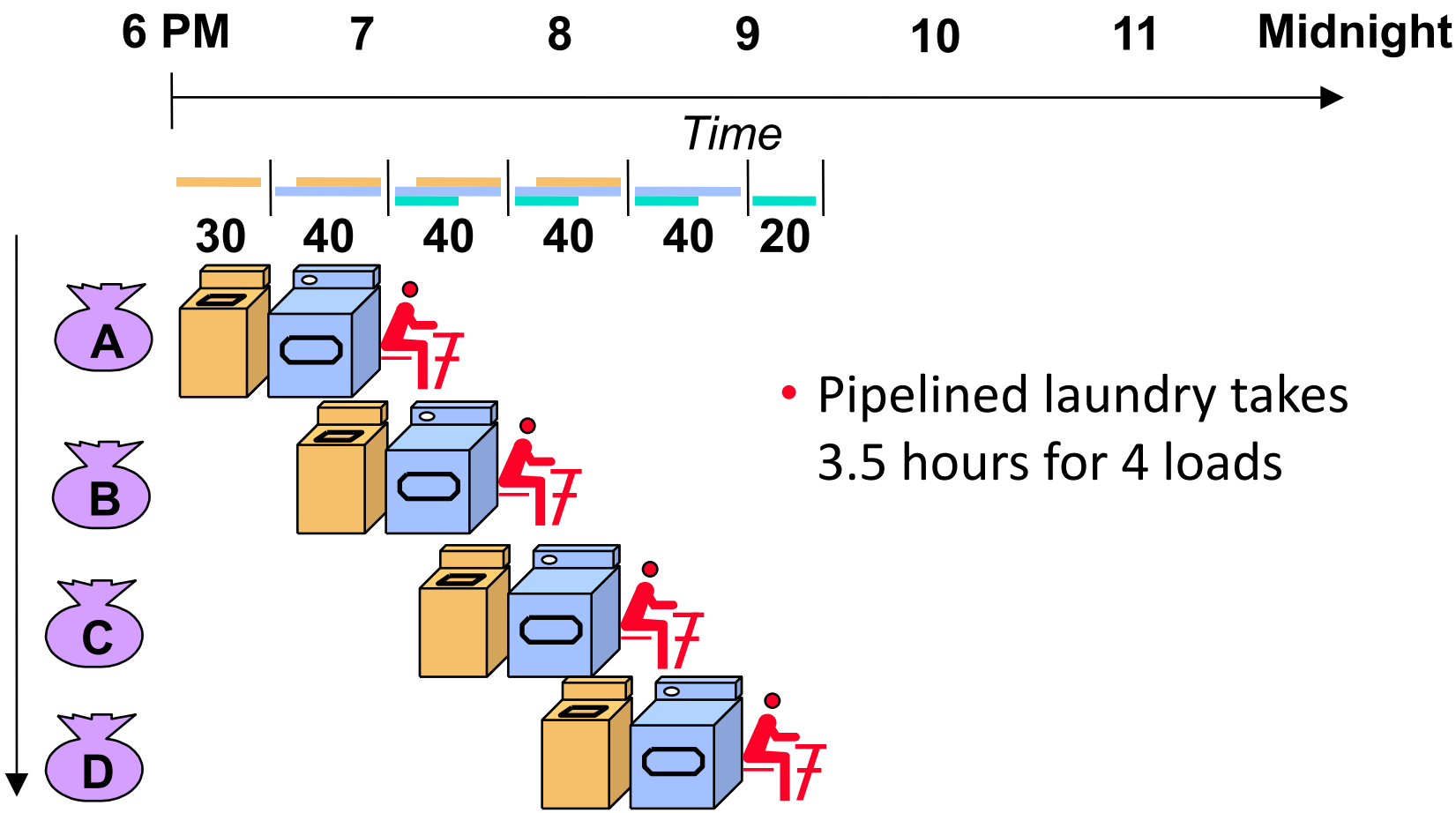
Traditional Pipeline Concept (2)

- Washer takes 30 minutes
- Dryer takes 40 minutes
- Folder takes 20 minutes



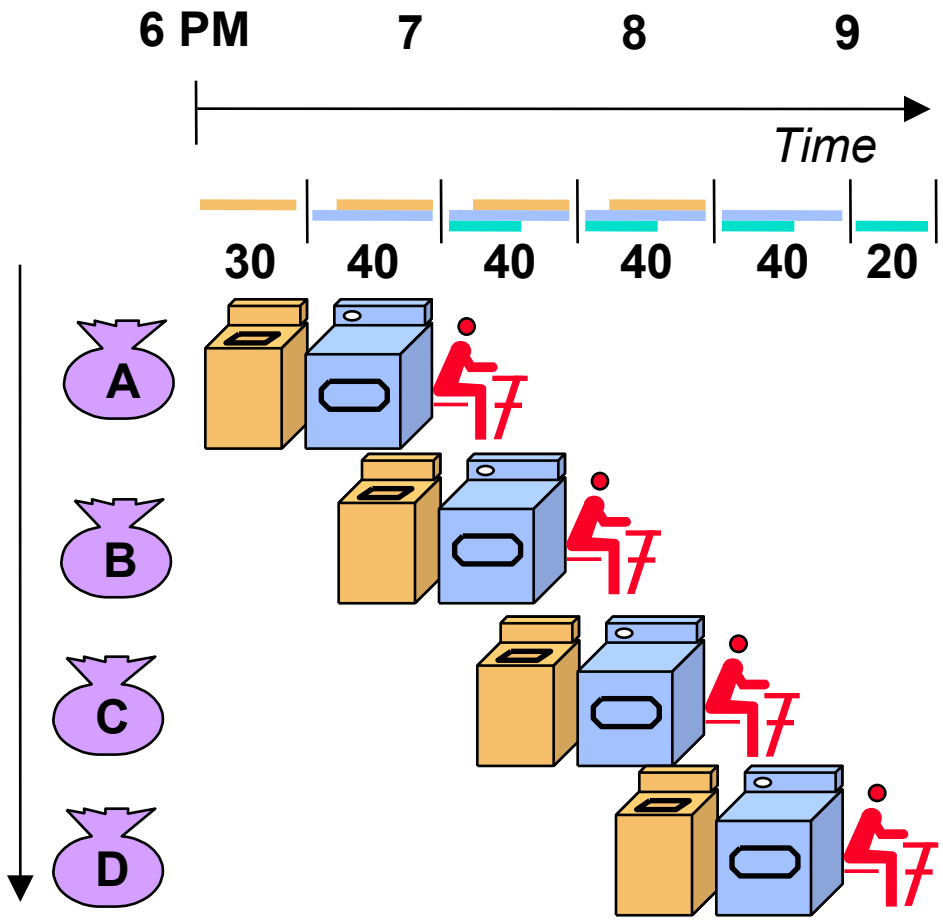
- Sequential laundry takes 6 hours for 4 loads
- If they learned pipelining, how long would laundry take?

Traditional Pipeline Concept (3)



- Pipelined laundry takes 3.5 hours for 4 loads

Traditional Pipeline Concept (4)



- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and time to "drain" it reduces speedup

- Multiple tasks operating simultaneously using different resources

Basic Concept

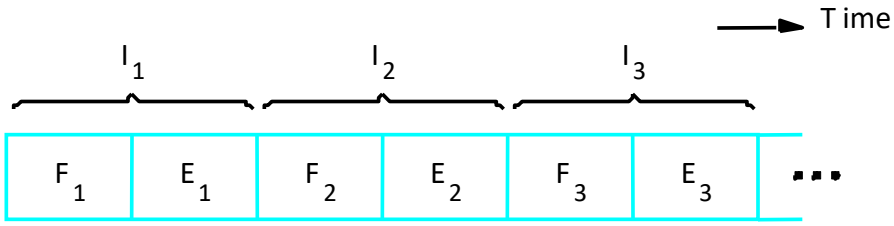
- Pipelining is a method that can be use to increase the execution speed.
- Pipelining arranges hardware in a way that more than one operation can be executed at the same time.
- The time used to complete a single instruction does not change.
- However, the number of instructions executed per second can be increased.

A Two-stage Pipeline (1)

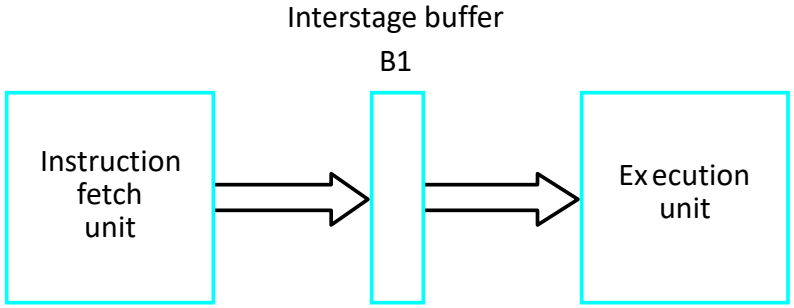
- For a two-stage pipeline, the execution of one instruction can be divided into two parts:
  - Fetch (F_i)
  - Execute (E_i)
- The fetching and the execution hardware can be separated into two different modules.
- For a multiple-instruction program, the execution time may be cut down by 50%.

A Two-stage Pipeline (2)

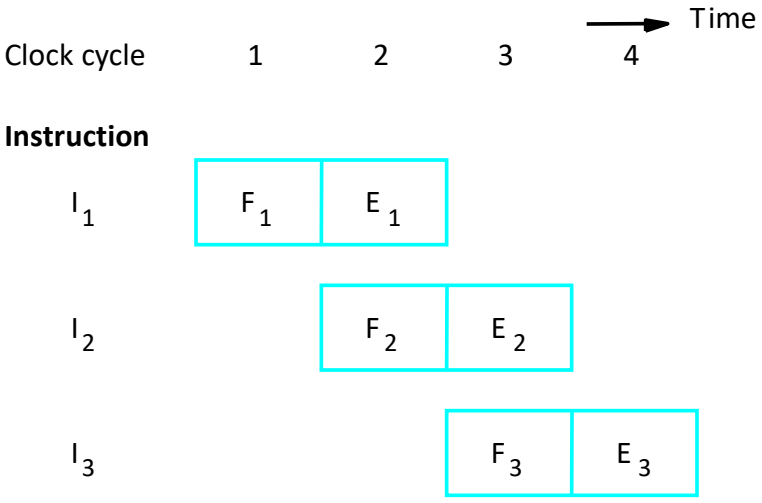
Fetch + Execution



(a) Sequential execution



(b) Hardware organization



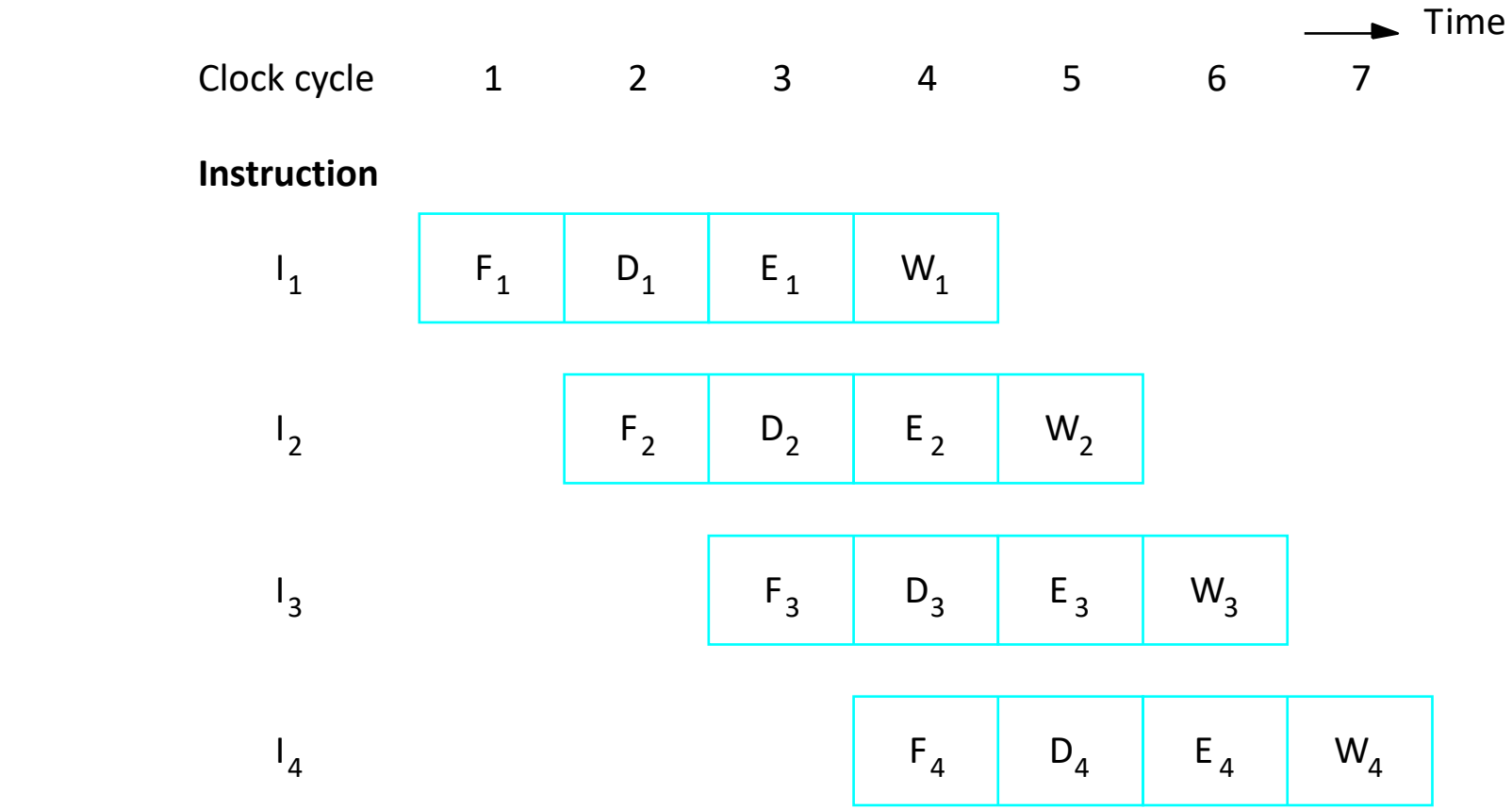
(c) Pipelined execution

A Four-stage Pipeline (1)

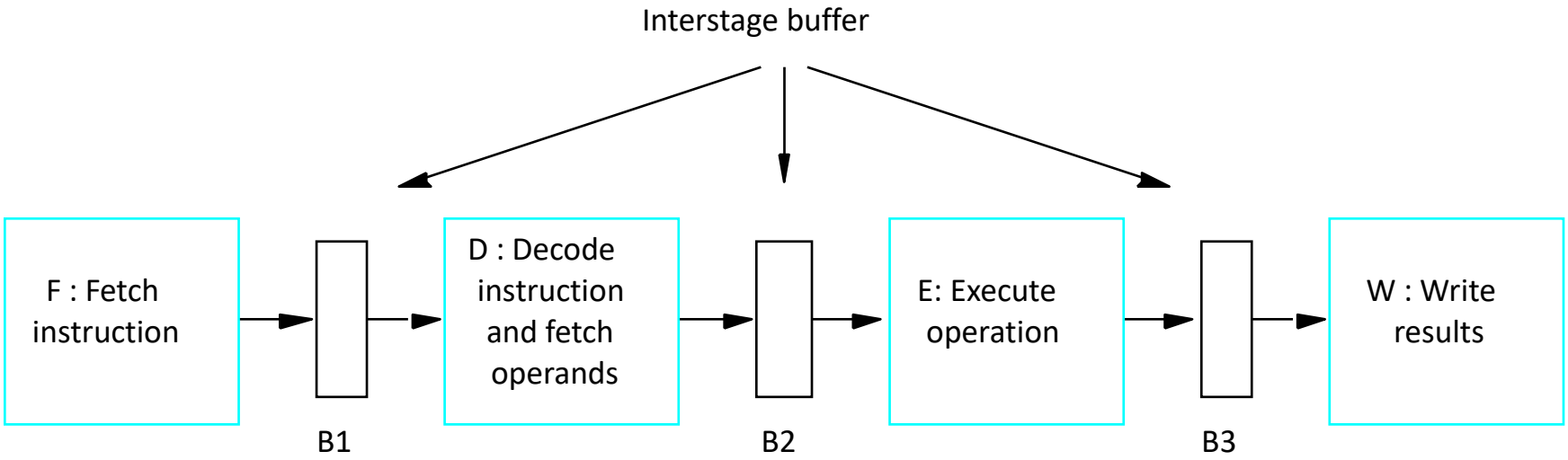
- An execution of an instruction can be divided into four stages as follows:
  - Fetch an instruction (from memory)
  - Decode an instruction and load operands
  - Execute an instruction
  - Write the output of an instruction

A Four-stage Pipeline (2)

Fetch + Decode + Execution + Write

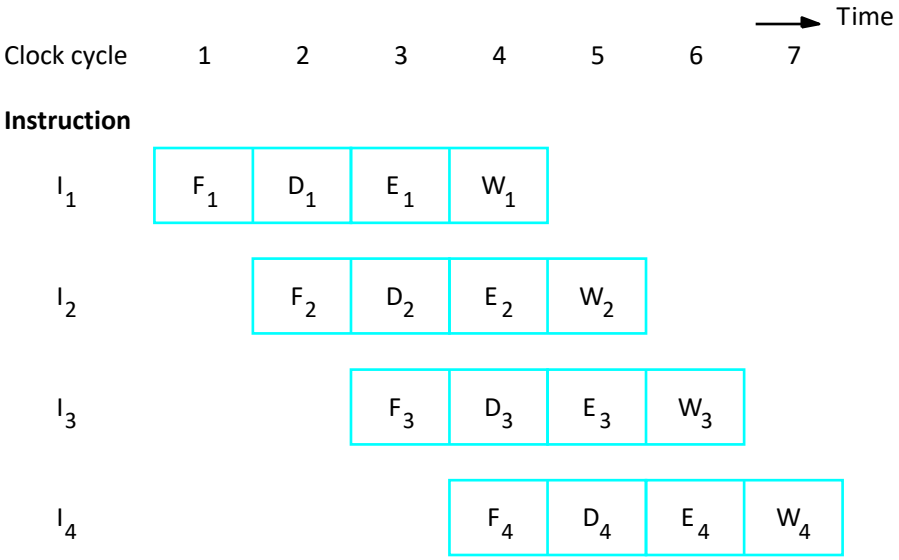


A Four-stage Pipeline (3)

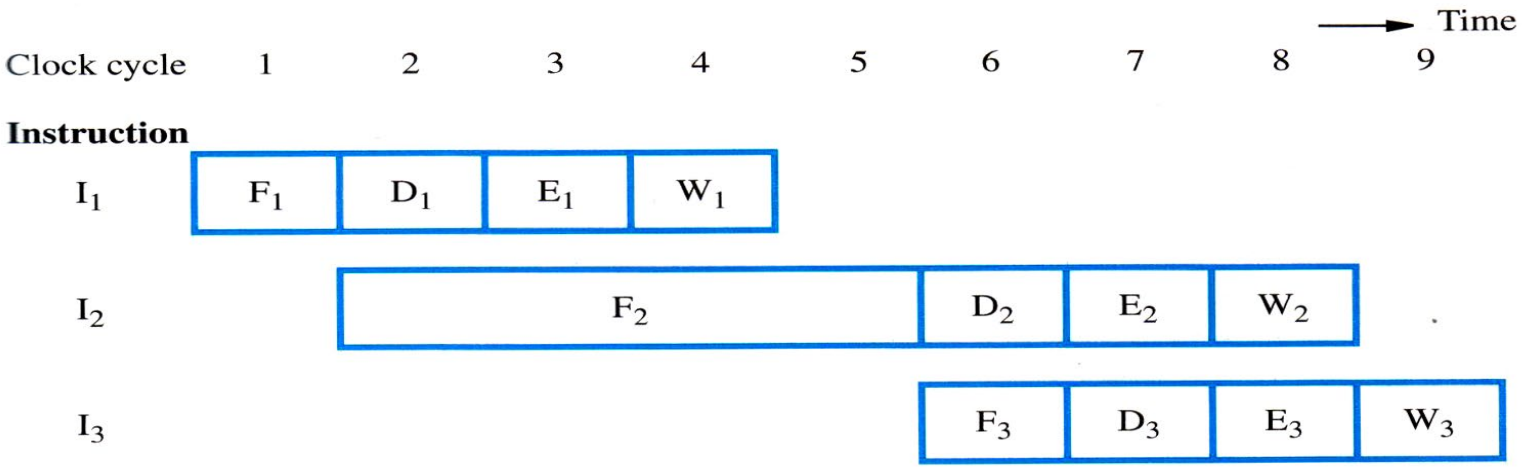


The Buffer

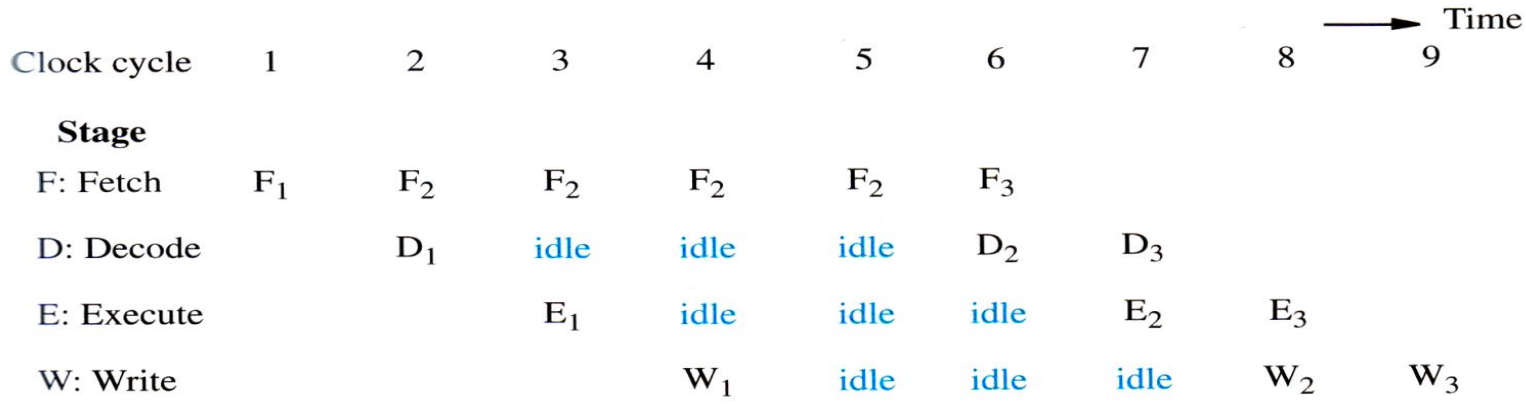
- At the beginning of the 4th clock:
  - B1 will hold I3, which is about to be decoded.
  - B2 will hold an operand from decoding stage (D). I2 is to be executed.
  - B3 will hold the result from the execution of I1, which will be written to memory.
  - I4 is to be fetched



Stall Cycles (1)



(a) Instruction execution steps in successive clock cycles

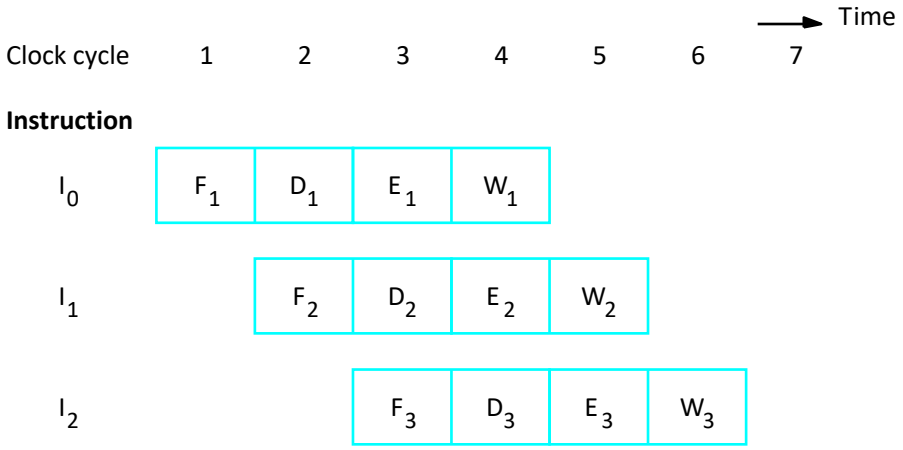


Stall Cycles (2)

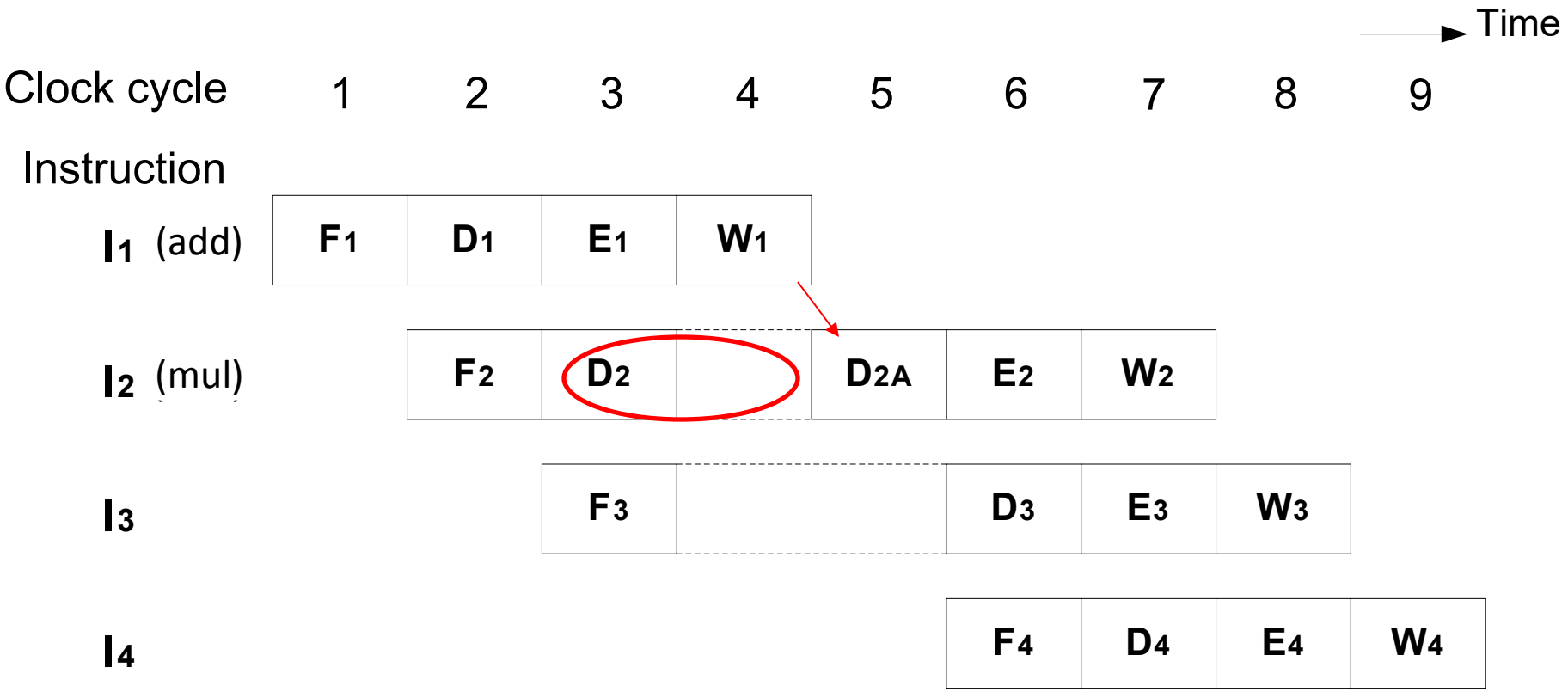
- Idle periods caused by:
  - Data hazard : data or operands are not available due to delayed-operations in a pipeline.
  - Control hazard : an instruction that is about to be fetched is not in a cache or a delay caused by a branch instruction.
  - Structural hazard : two instructions need to use the same hardware at the same time.

Data Hazard

- The pipeline is stalled because the requested data is not available. For example,
  - I_0 A = 5
  - I_1 A = A+3
  - I_2 B = A*4
- If I1 and I2 are executed sequentially B=32
- If I1 and I2 are executed simultaneously B=20



Data Hazard (2)

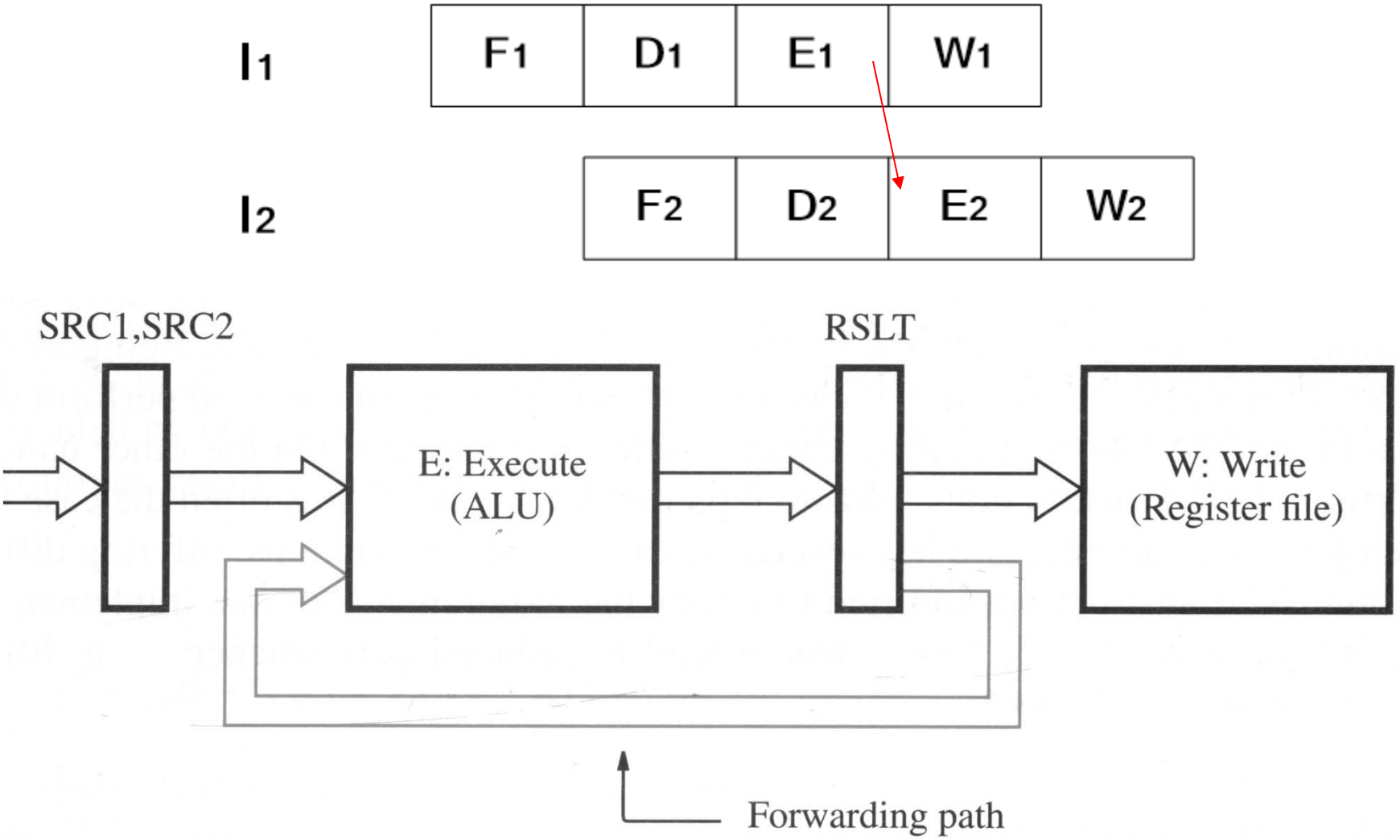


Pipeline is stalled by data dependency between D_2 and W_1

Operand forwarding

- From the previous example:
  - Station E2 has to wait for I1 to write the result to the register before it can read.
  - In fact, the result of I1 is available right after completion of station E1.
  - Thus, we can forward the execution result to E2 station to reduce the number of stall cycle.

Operand forwarding (2)



Data dependency

Data dependency can be eliminated using a smart compiler. More efficient code can increase the pipeline efficiency.

Loop Unrolling

```
for (i=0;i<n;i++)
    A[i] += B[i]+C[i]
```

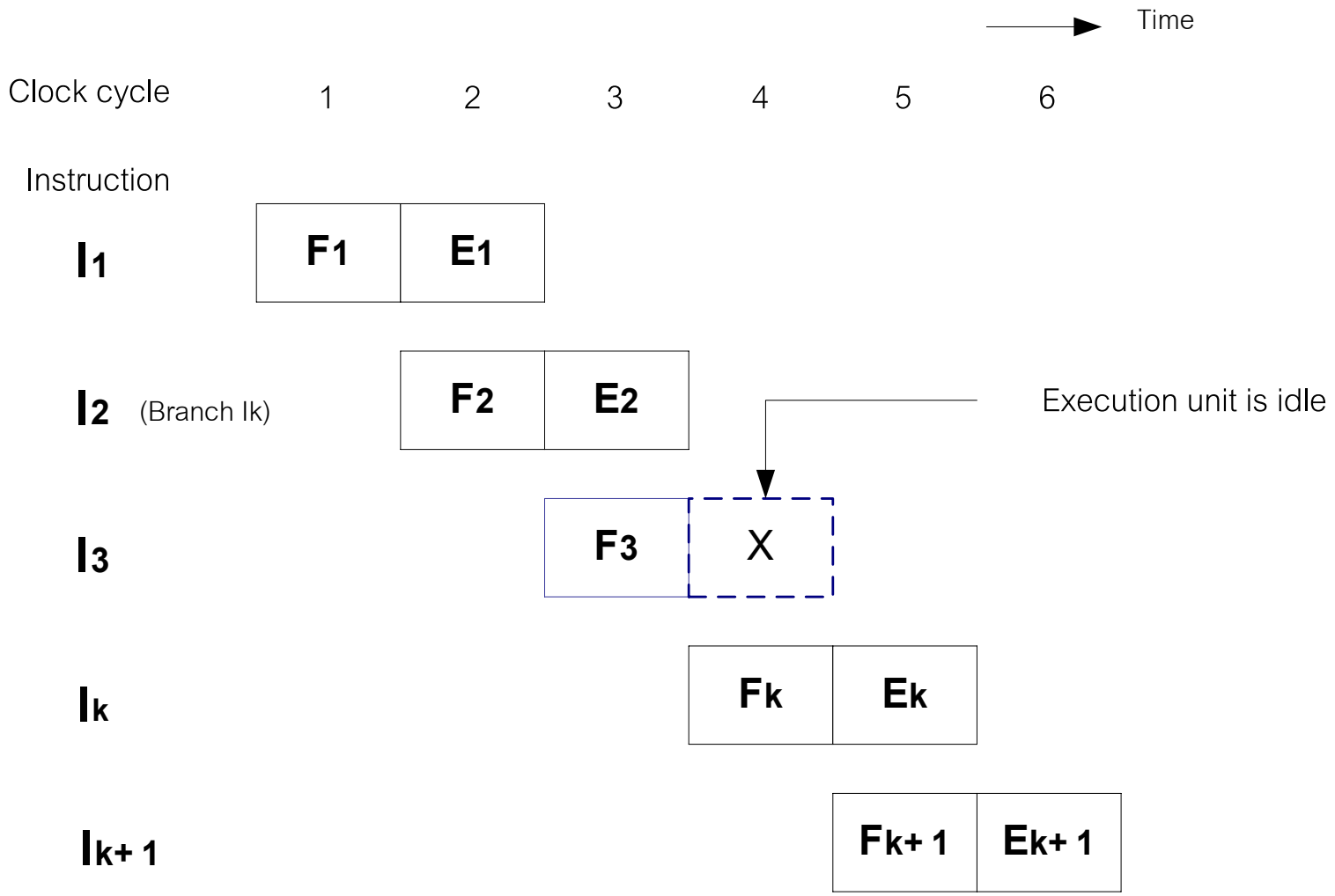
```
MOV A[i] ,R1
ADD B[i] ,R1
ADD C[i] ,R1
```

=>

```
for (i=0;i<n; i+=2)
    A[i] += ....
    A[i+1] += ....
```

```
MOV A[i] ,R1
MOV A[i+1] ,R2
ADD B[i] ,R1
ADD B[i+1] ,R2
ADD C[i] ,R1
ADD C[i+1] ,R2
```

Control Hazard

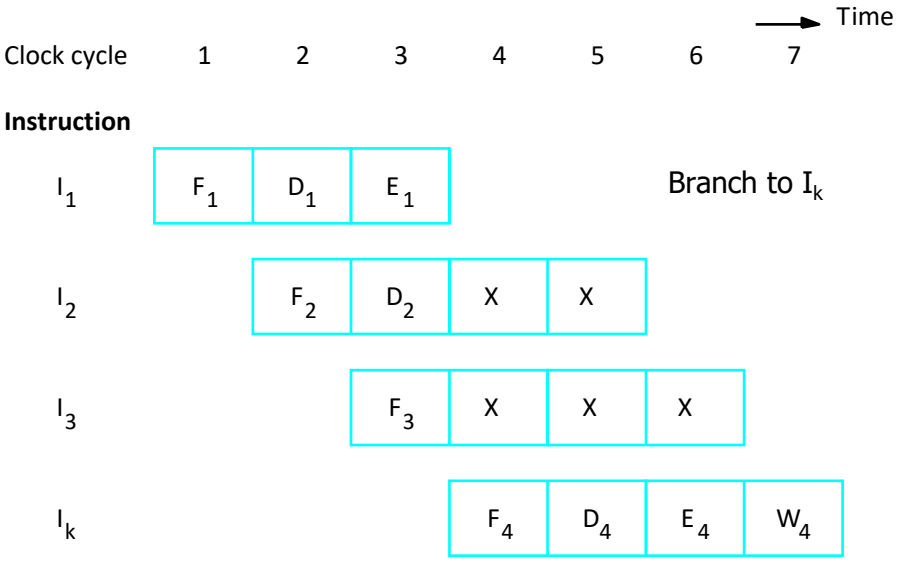


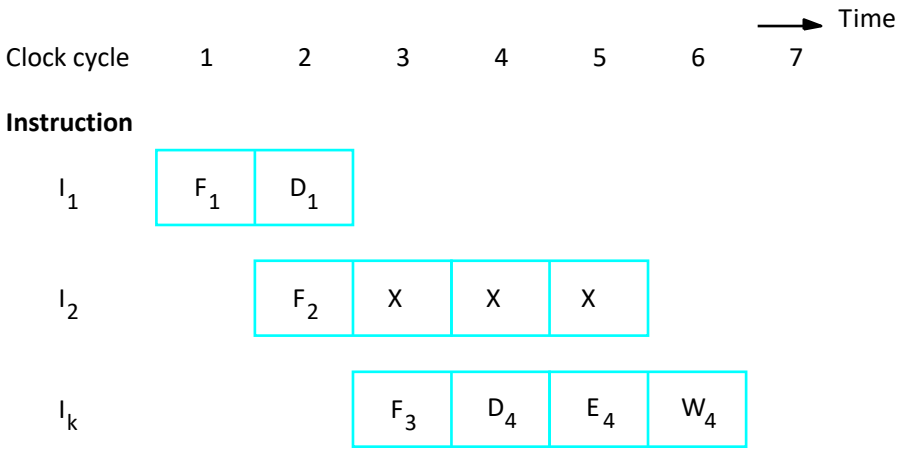
Unconditional Branch

- I2 is a branch instruction with the destination being Ik. During the third cycle, I3 is fetched and I2 is executed and the branch target is computed. In the fourth cycle, I3 has to be thrown away causing 1 stall cycle.

Reducing Branch Penalty

- The target address is determined in the Decode state





Conditional Branch
(if-then-else)

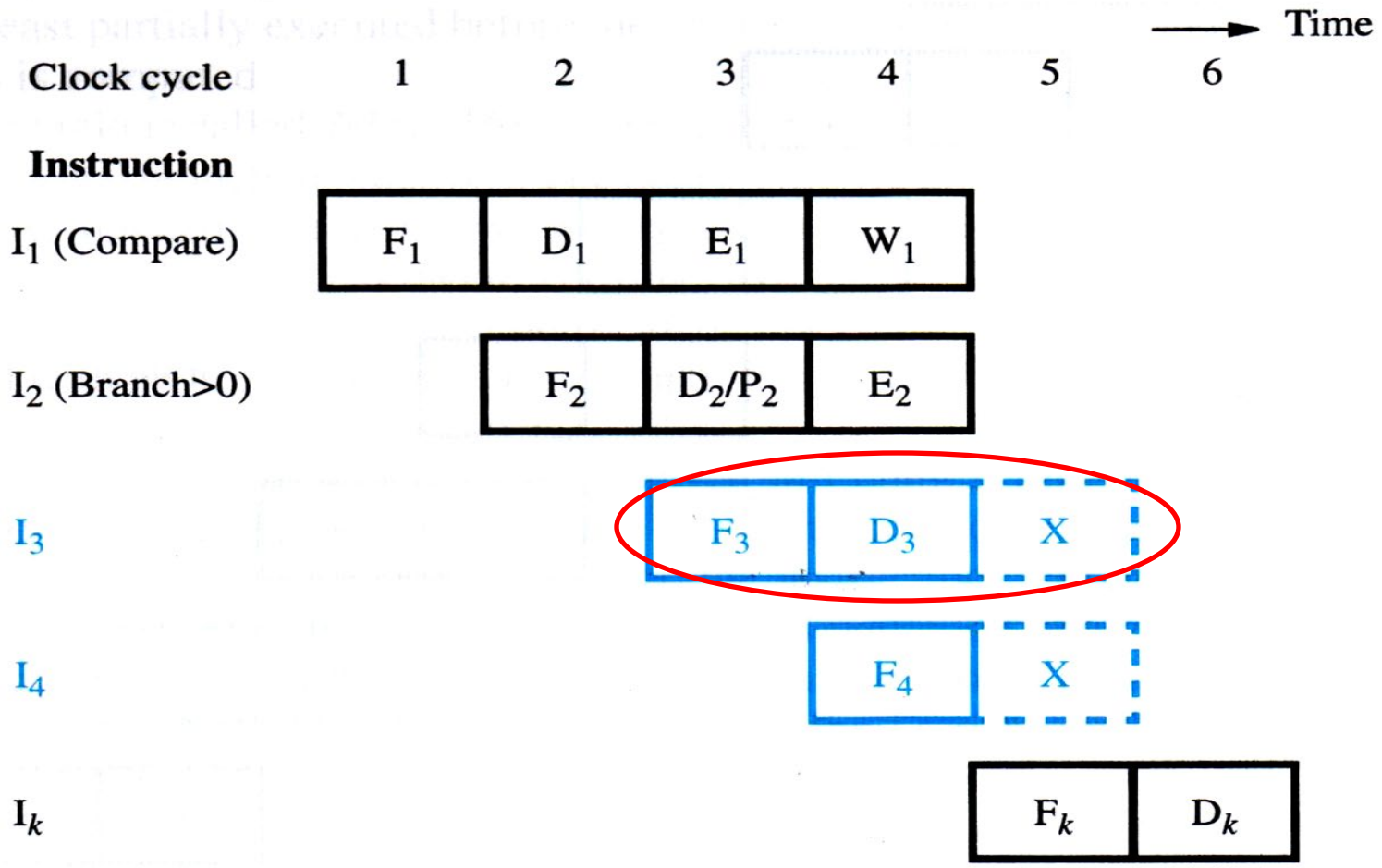
- The branch target will only be available after the execution of the branch instruction.
- Introduce more stall cycles than the unconditional branch instructions.
- Reduce branch penalty by
  - The branch delay slot
  - The branch prediction

The Branch Delay Slot (1)

P1:			P2:	
Loop	Shift-left R1		Loop	Dec R2
	Dec R2	=>		Br=0 Loop
	Br=0 Loop			Shift-left R1
Next	Add R1,R3		Next	Add R1,R3

If the instruction sequence can be swapped,
rearranging codes can reduce branch delay slot.
For example, pipeline P2 produces the same result as P1, with no delay slot.

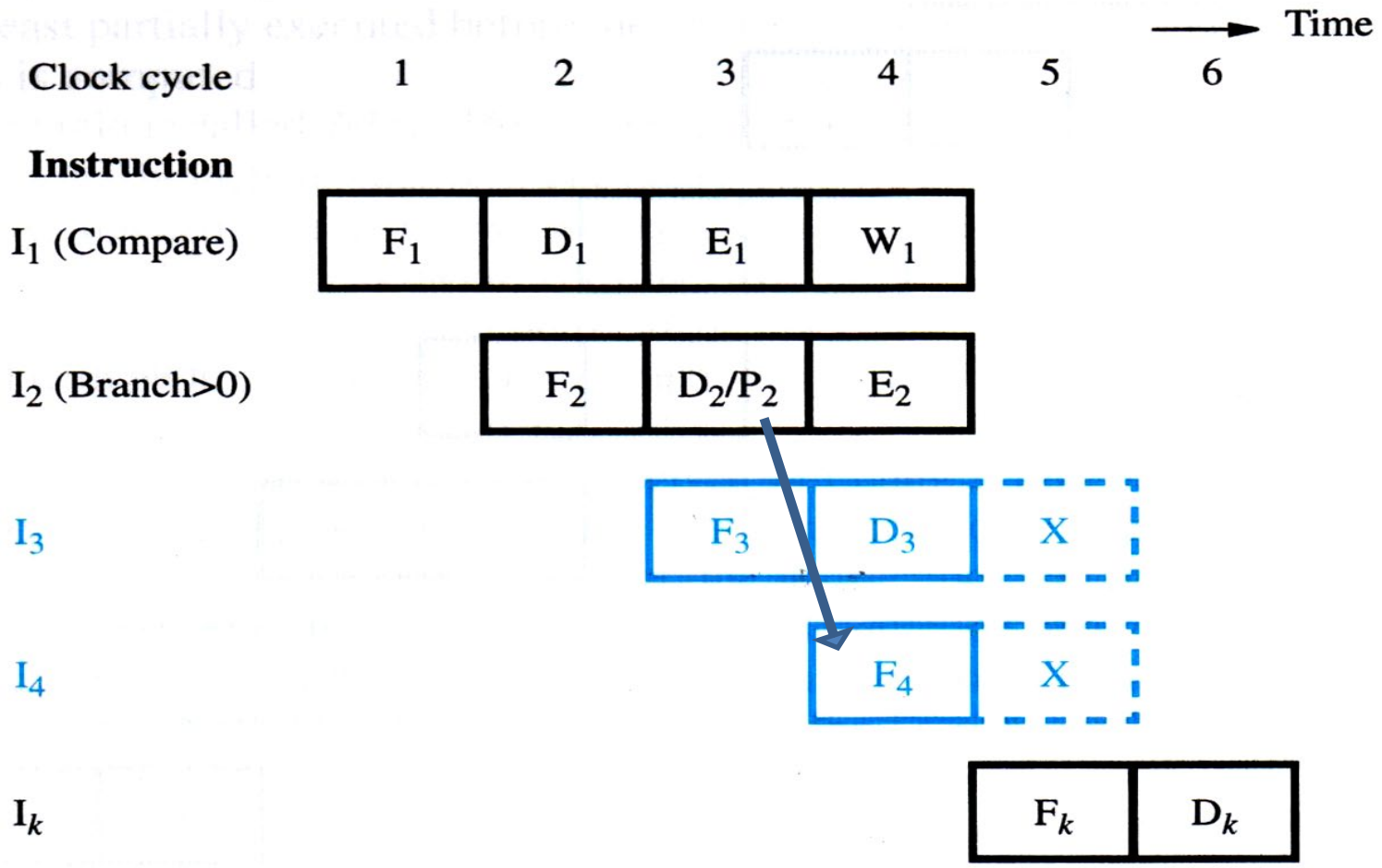
The Branch Delay Slot(2)



Static Branch Prediction (1)

- For static branch prediction should be made based on the behavior of the program. For example, in a loop structure, we can assume that branch will be true all the time except for the last iteration.
- Determine the static predication of taken or not-taken by checking the sign of branch offset.
- Predict whether a branch will take place, and fetch the next instruction.

Static Branch Prediction (2)

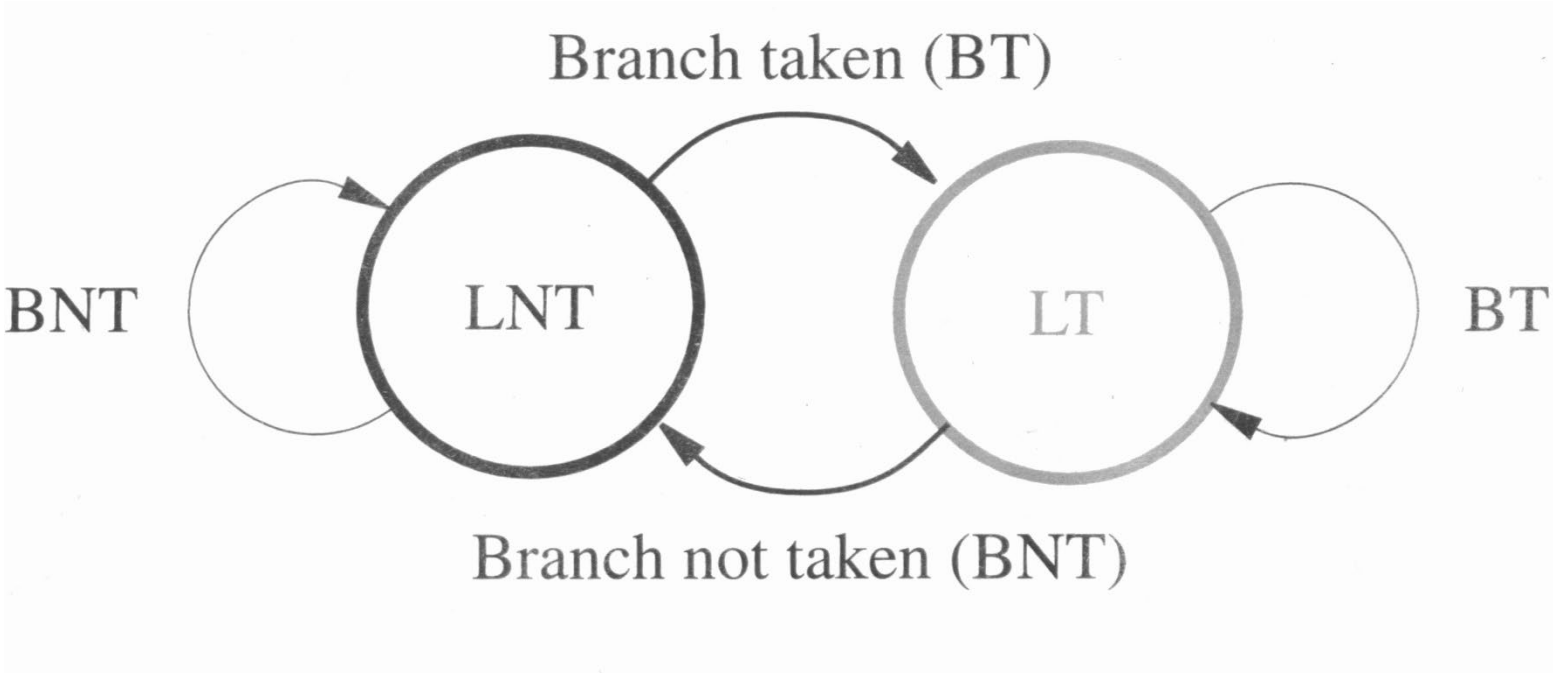


Dynamic Branch Prediction

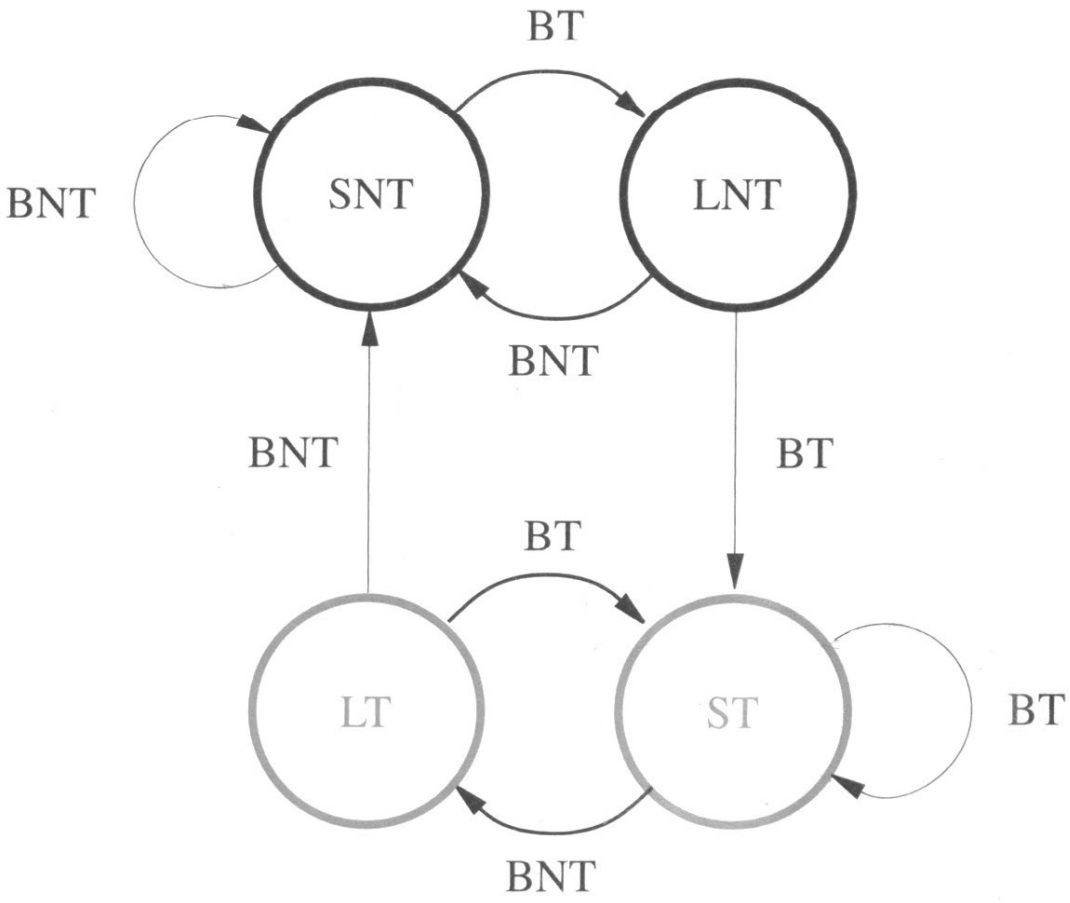
- Processor hardware will store branch decisions for use in the next prediction. (Using history to predict the future)
- State machine: processor only need one bit to store history information.

2-state Prediction

- LT is branch taken
- LNT is branch not taken

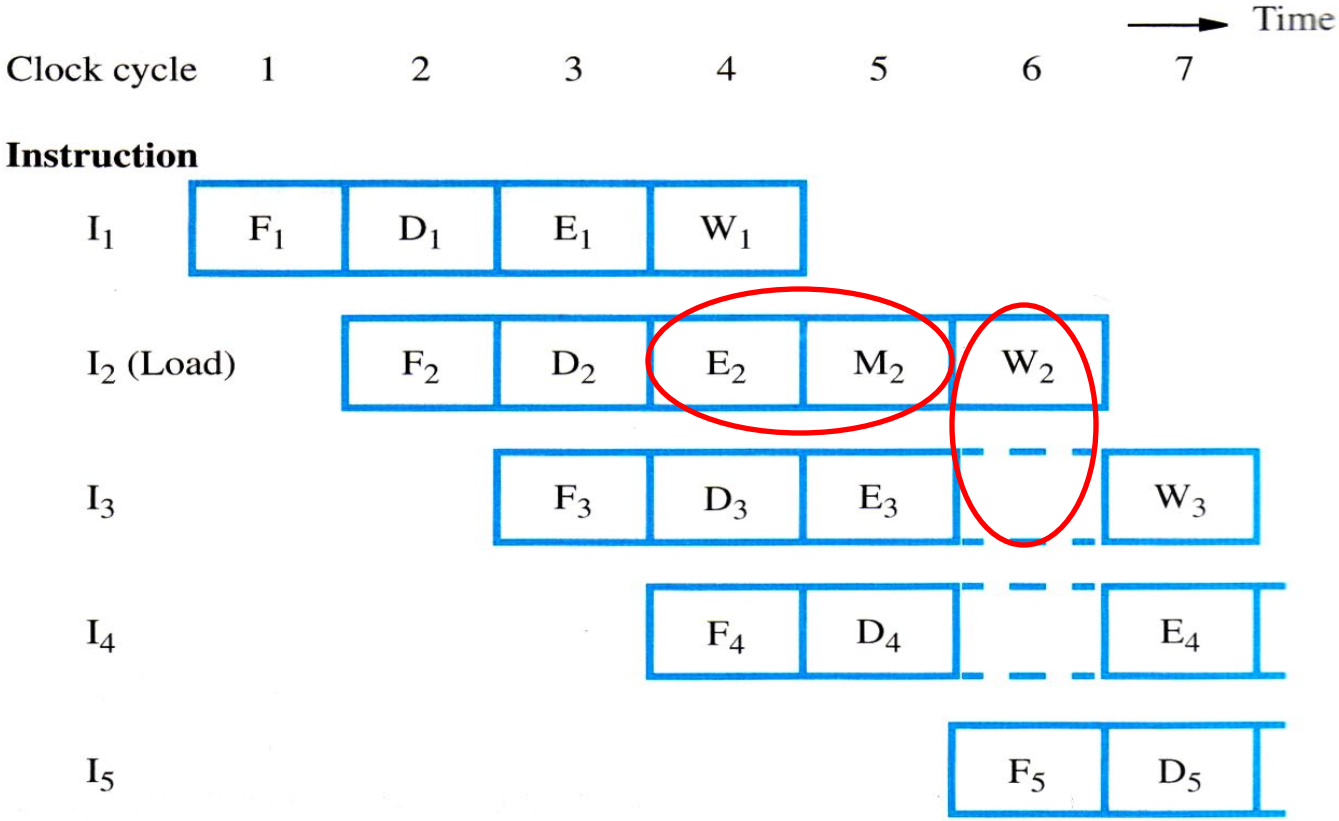


4-state Prediction



- ST = strongly likely to be taken
- LT = likely to be taken
- LNT = likely not to be taken
- SNT = strongly likely not To be taken

Structural Hazard



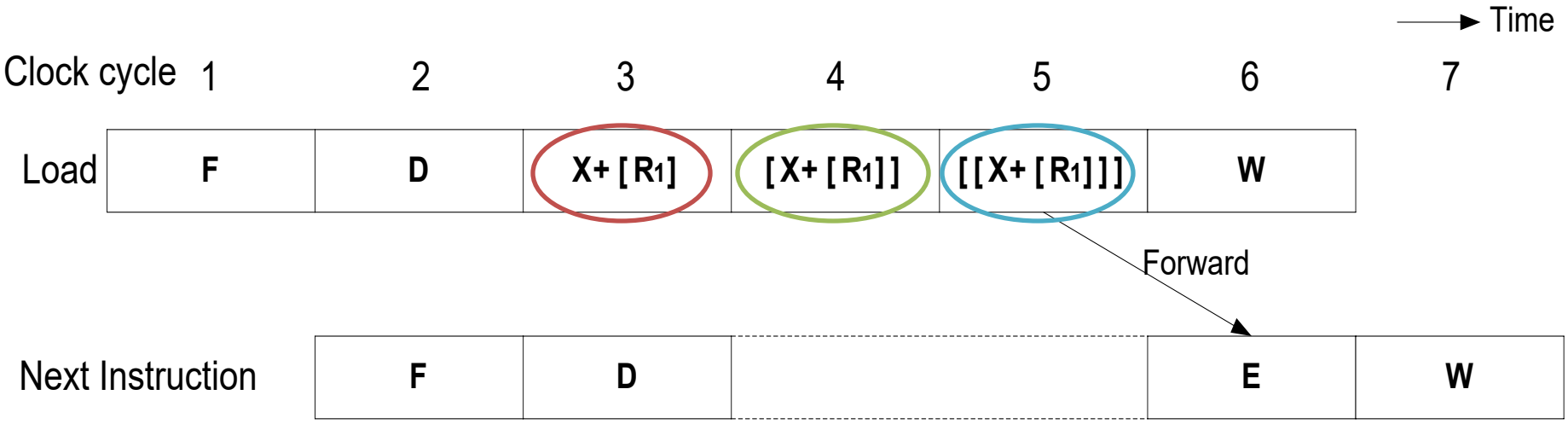
Two instructions request the same hardware

At the 6th cycle, I3 cannot write because I2 also requests the write operation.

Addressing Mode

- Instruction flow of different addressing mode can affect the pipeline.
- Complex addressing mode can cause stall cycles. For example,

Load (X(R1)), R2



Addressing Mode (2)

- Using complex addressing mode, the number of instructions may be reduced, but not the execution time. For example, "Load (X(R1)), R2", can be rewritten as follows:

  Add #X, R1, R2

  Load (R2), R2

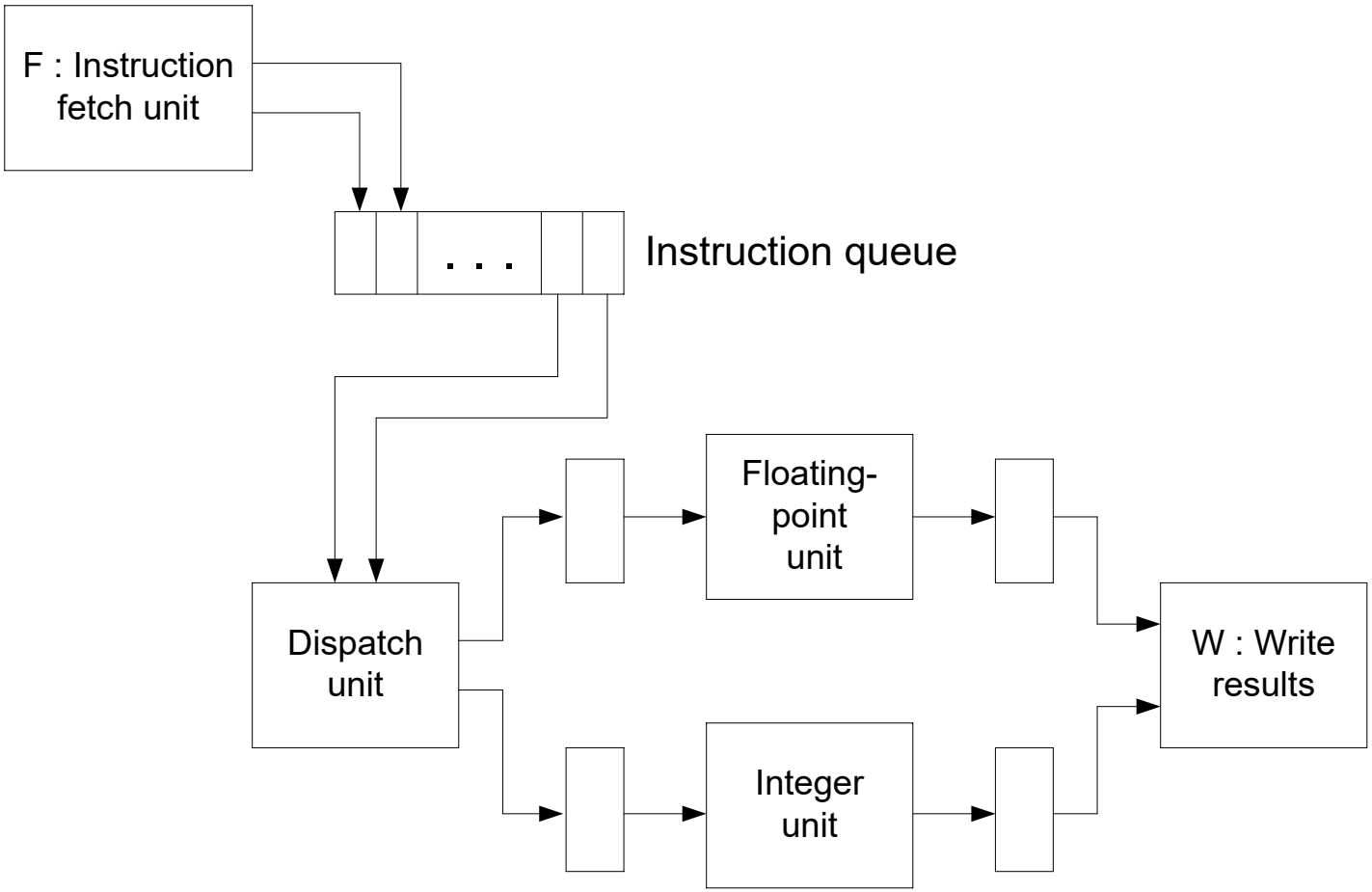
  Load (R2), R2

  The new set of instructions also use 6 clock cycles without introducing stall cycles. Thus, it is preferable in many new processor.

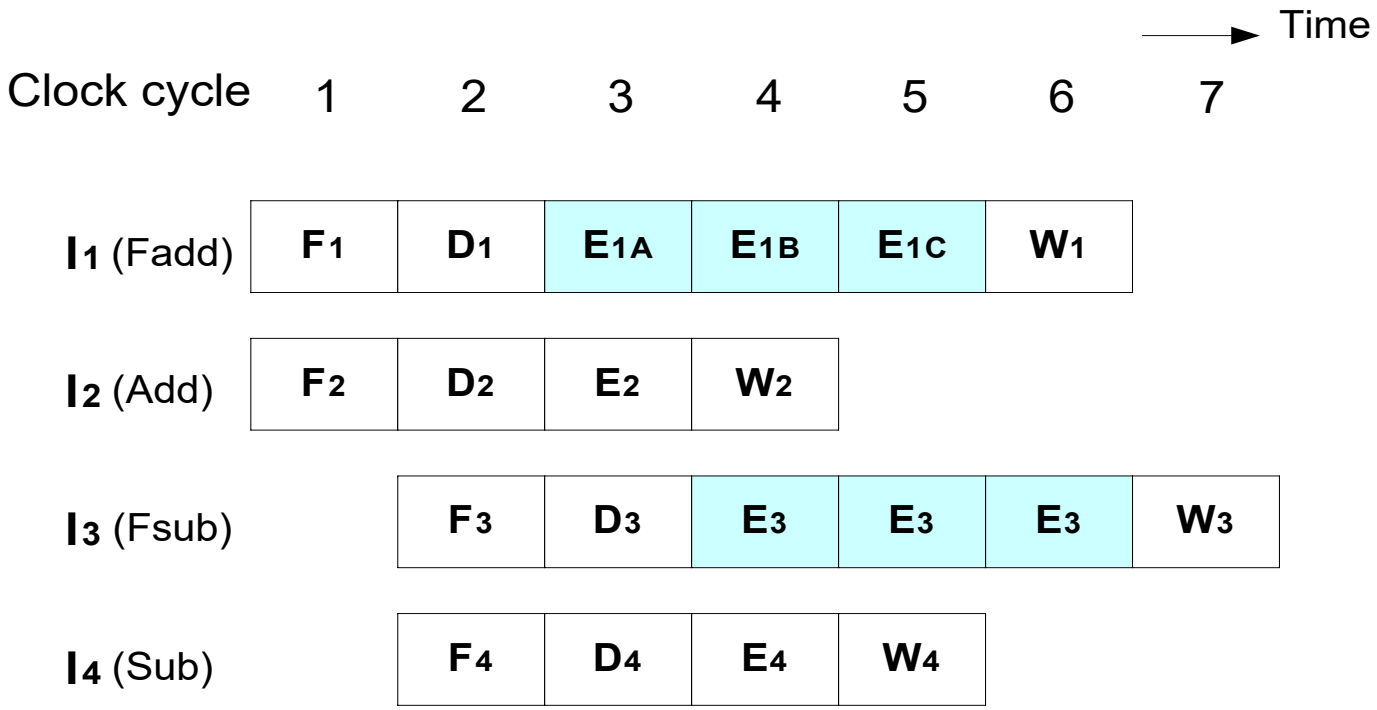
Superscalar Operation

- Superscalar processor is the processor that has multiple processing units.
- The processor can handle more than one instruction at a time.
- Throughput can be more than one instruction/cycle.

Superscalar Operation (2)



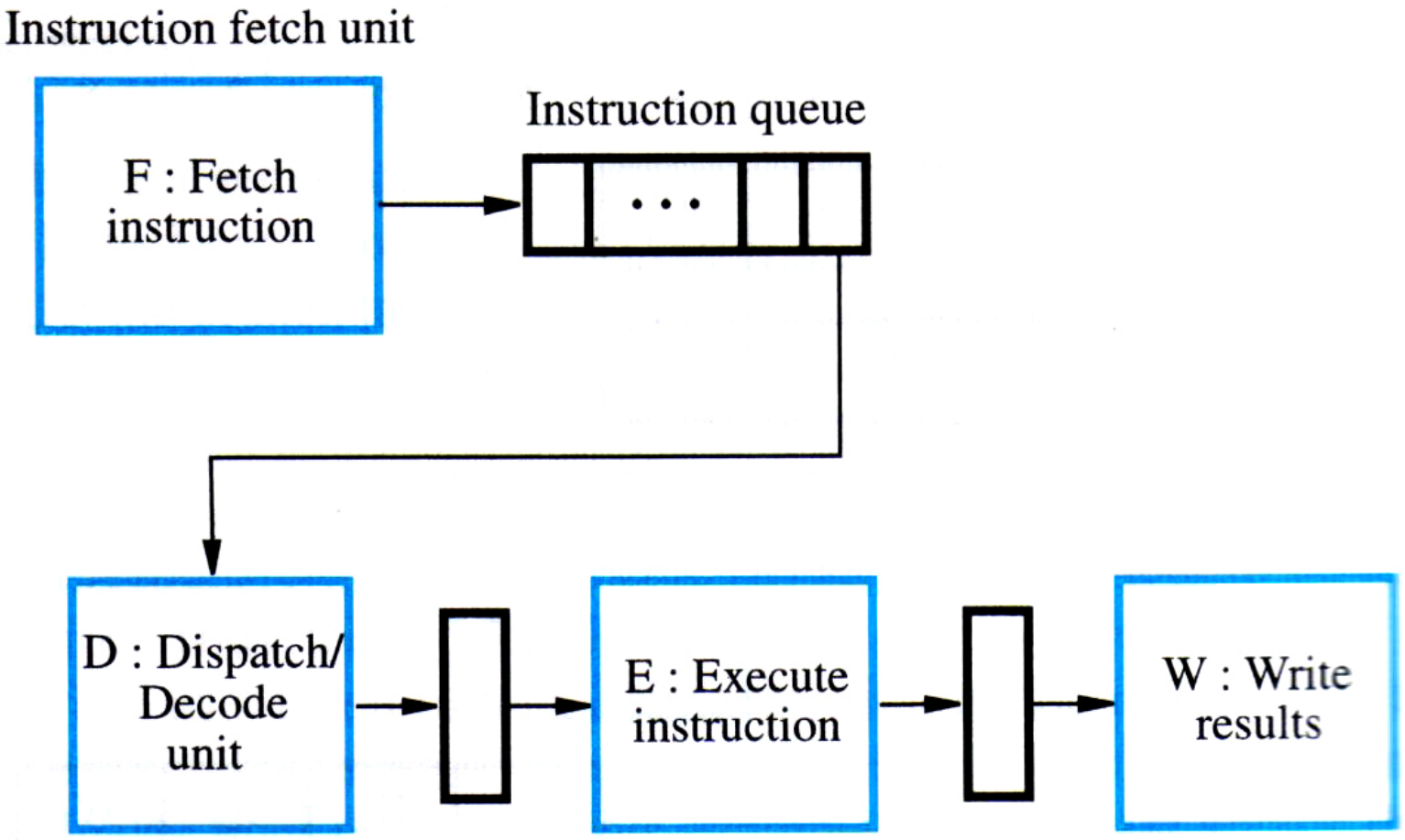
Superscalar Operation (3)



- If I2 depends on I1, delay must be inserted, anyway.
- If I1 caught exception, the program will be enter an inconsistent state (because I2 have done W2, but I1 got an exception before W1, ex. divide by 0).

Supplementary Section

Instruction Queue and Pre-fetching



Performance

- Instruction throughput for sequential execution.

$$Ps = R/S$$

Let: Ps is throughput

R is Clock rate

S is number of clock cycle/instruction

Thus, an *n*-stage pipeline has a possibility of increasing the throughput up to *n* time.